# Design Patterns – Quick Study Notes (GoF)

These notes provide concise explanations of the 23 Gang of Four design patterns. Each pattern includes a precise definition, structure components, explanation, and common real-world uses. Useful for quick revision for Low-Level Design (LLD) interviews and backend engineering preparation.

# Creational Patterns

## Singleton

**Definition:** Ensures a class has only one instance and provides a global access point to it.

**Explanation:**

- Single instance shared across application
- Private constructor prevents direct creation
- Static getInstance() method provides access

**Common Uses:**

- Logging systems
- Configuration manager
- Database connection manager
- Java Runtime.getRuntime()

## Factory Method

**Definition:** Defines an interface for creating objects but lets subclasses decide which class to instantiate.

**Explanation:**

- Encapsulates object creation
- Client code depends on abstraction, not concrete classes
- Subclasses choose which object to create

**Common Uses:**

- LoggerFactory in frameworks
- Payment processor selection
- Calendar.getInstance() in Java

## Abstract Factory

**Definition:** Provides an interface to create families of related objects without specifying concrete classes.

**Explanation:**

- Creates related objects together
- Ensures compatibility between products
- Factory of factories concept

**Common Uses:**

- UI toolkits (Windows/Mac components)

- Database driver families
- Theme-based UI systems

## Builder

**Definition:** Separates construction of a complex object from its representation so the same process can create different representations.

**Explanation:**

- Object built step-by-step
- Fluent APIs often use builder pattern
- Optional Director controls build order

**Common Uses:**

- HTTP request builders
- StringBuilder
- Lombok @Builder
- OkHttp Request.Builder

## Prototype

**Definition:** Creates new objects by cloning an existing instance instead of creating them from scratch.

**Explanation:**

- Uses clone operation
- Useful when object creation is expensive
- Supports deep or shallow copy

**Common Uses:**

- Game object duplication
- Document templates
- Spring prototype scope
- Configuration cloning

# Structural Patterns

## Adapter

**Definition:** Converts one interface into another that a client expects.

**Explanation:**

- Acts as bridge between incompatible interfaces
- Allows legacy code integration
- Wraps existing class

**Common Uses:**

- Payment gateway adapters
- InputStreamReader in Java

- Legacy API integration

## Bridge

**Definition:** Separates abstraction from implementation so they can evolve independently.

**Explanation:**

- Two hierarchies: abstraction and implementation
- Reduces class explosion from combinations

**Common Uses:**

- Remote control devices controlling different appliances
- Cross-platform UI frameworks

## Composite

**Definition:** Composes objects into tree structures so individual objects and groups are treated uniformly.

**Explanation:**

- Represents part-whole hierarchy
- Leaf and Composite share same interface

**Common Uses:**

- File systems
- Organization hierarchy
- UI component trees

## Decorator

**Definition:** Adds responsibilities to objects dynamically without modifying the original class.

**Explanation:**

- Wraps original object
- Multiple decorators can stack behaviors

**Common Uses:**

- Java I/O streams
- Middleware pipelines
- Logging wrappers

## Facade

**Definition:** Provides a simplified interface to a complex subsystem.

**Explanation:**

- Hides internal complexity
- Centralized entry point

**Common Uses:**

- Home theater systems

- Spring JdbcTemplate
- API gateway wrappers

## Flyweight

**Definition:** Reduces memory usage by sharing common objects.

**Explanation:**

- Stores intrinsic state in shared object
- External state passed during usage

**Common Uses:**

- Text editor character sharing
- Game engines with many objects
- Java String pool

## Proxy

**Definition:** Provides a placeholder controlling access to a real object.

**Explanation:**

- Adds access control, caching, or lazy loading
- Proxy implements same interface as real object

**Common Uses:**

- Hibernate lazy loading
- Security proxies
- Remote service proxies

# Behavioral Patterns

## Chain of Responsibility

**Definition:** Passes a request along a chain of handlers until one processes it.

**Explanation:**

- Handlers linked sequentially
- Each handler decides to process or forward request

**Common Uses:**

- Servlet filters
- Authentication pipelines
- Logging handlers

## Command

**Definition:** Encapsulates a request as an object allowing parameterization and queuing.

**Explanation:**

- Separates requester from executor
- Supports undo/redo

**Common Uses:**

- Task queues
- Remote controls
- GUI button actions

## Interpreter

**Definition:** Defines a grammar and interpreter for evaluating language expressions.

**Explanation:**

- Represents grammar rules as classes
- Processes expressions recursively

**Common Uses:**

- SQL interpreters
- Regular expression engines
- Compilers

## Iterator

**Definition:** Provides sequential access to elements of a collection without exposing its structure.

**Explanation:**

- Separate traversal logic
- Common in collection frameworks

**Common Uses:**

- Java Collection iterators
- Database cursor traversal

## Mediator

**Definition:** Centralizes communication between multiple objects to reduce dependencies.

**Explanation:**

- Objects communicate through mediator instead of directly

**Common Uses:**

- Chat room coordination
- Air traffic control systems

## Memento

**Definition:** Captures and restores an object's internal state without exposing its details.

**Explanation:**

- Stores snapshot of object state

- Supports undo functionality

**Common Uses:**

- Text editor undo
- Game save checkpoints

## Observer

**Definition:** Defines a one-to-many dependency so that when one object changes state, all dependents are notified.

**Explanation:**

- Publisher-subscriber relationship
- Loose coupling between components

**Common Uses:**

- Event listeners
- Stock price notifications
- Kafka consumers

## State

**Definition:** Allows an object to change its behavior when its internal state changes.

**Explanation:**

- State objects encapsulate behavior
- Context delegates behavior to current state

**Common Uses:**

- Order lifecycle systems
- Traffic light control

## Strategy

**Definition:** Defines a family of algorithms and allows switching between them at runtime.

**Explanation:**

- Encapsulates algorithms
- Context chooses strategy dynamically

**Common Uses:**

- Payment methods
- Sorting algorithms
- Compression strategies

## Template Method

**Definition:** Defines the skeleton of an algorithm while allowing subclasses to override specific steps.

**Explanation:**

- Base class defines algorithm structure
- Subclasses customize certain steps

**Common Uses:**

- Spring JdbcTemplate workflow
- Data processing frameworks

## Visitor

**Definition:** Separates algorithms from object structures so new operations can be added easily.

**Explanation:**

- Visitor performs operations on elements
- Elements accept visitors

**Common Uses:**

- Compiler AST processing
- File system analytics tools